

```

"""
test_o2v4_replication.py
=====

Port fidèle de O2v4_phasel.py pour audit externe.
Objectifs :
    1. Reproduire  $d_H \approx 3.110 \pm 0.156$  à  $N=500$  (réf JSON Rudolphe)
    2. Confirmer que l'écart précédent ( $d_h = 1.78$ ) était purement méthodologique
    3. Bonus : tester eigenvalue-counting  $d_s$  comme méthode alternative
        (la méthode plateau heat-kernel d'O2v4 échoue partout)
"""

import numpy as np
import math, time
from collections import deque
from scipy.spatial import KDTree
from scipy.linalg import eigvalsh
from scipy.stats import linregress

# Config V4 (frozen, identique à O2v4_phasel.py)
CFG = {
    "target_degree": 8.0,
    "proposal_radius_mult": 1.5,
    "burn_in_factor": 100, # Phase A
    "prod_factor": 200, # Phase A
    "tau": 0.50,
    "tau_scale_N": True,
    "beta": 0.30,
    "dH_n_sources": 10,
    "dH_min_sources": 5,
    "dH_r_min_fit": 2,
    "dH_inner_fraction": 0.70,
    "dH_min_ball_count": 3,
    "min_lcc_ratio": 0.90,
}

def _rips_radius(N, cfg=CFG):
    k = cfg["target_degree"]
    return (k / ((N-1) * (4/3) * math.pi)) ** (1/3)

def _E0(N, cfg=CFG):
    return int(N * cfg["target_degree"] / 2)

def _lcc_size(adj, N):
    visited = [False]*N

```

```

best = 0
for s in range(N):
    if not visited[s]:
        sz = 0; q = deque([s]); visited[s] = True
        while q:
            v = q.popleft(); sz += 1
            for w in adj[v]:
                if not visited[w]:
                    visited[w] = True; q.append(w)
        best = max(best, sz)
return best

def _extract_lcc(adj, N):
    visited = [False]*N; comps = []
    for s in range(N):
        if not visited[s]:
            comp = []; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); comp.append(v)
                for w in adj[v]:
                    if not visited[w]:
                        visited[w] = True; q.append(w)
            comps.append(comp)
    lcc = max(comps, key=len)
    return lcc, {v:k for k,v in enumerate(lcc)}, len(comps)

def _build_initial(N, seed, cfg=CFG):
    r = _rips_radius(N, cfg)
    r_p = cfg["proposal_radius_mult"] * r
    E0 = _E0(N, cfg)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K_curr = sum(len(adj[i]) for i in range(N)) // 2
    attempts = 0
    max_att = 10 * abs(E0 - K_curr) + 5000
    while K_curr != E0 and attempts < max_att:
        attempts += 1
        if K_curr < E0:
            e = pool[rng2.integers(len(pool))]
            u, v = e
            if v not in adj[u]:

```

```

        adj[u].add(v); adj[v].add(u); K_curr += 1
    else:
        edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
        if not edges: break
        e = edges[rng2.integers(len(edges))]
        u, v = e
        adj[u].discard(v); adj[v].discard(u)
        if _lcc_size(adj, N) == N:
            K_curr -= 1
        else:
            adj[u].add(v); adj[v].add(u)
    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S_canonical(adj, n_tri, N, mode, cfg=CFG):
    beta = cfg["beta"]
    degs = np.array([len(adj[i]) for i in range(N)], float)
    mu = degs.mean()
    D = degs.std() / (mu + 1e-8)
    DI = math.log(1 + n_tri)
    if mode == "blob": return DI - beta * D
    raise ValueError(mode)

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()
        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

def run_canonical_mcmc(pts, adj_init, pool, N, E0, mode, seed, cfg=CFG):
    tau = cfg["tau"] / N if cfg.get("tau_scale_N", False) else cfg["tau"]
    bf = cfg["burn_in_factor"]
    pf = cfg["prod_factor"]
    rng = np.random.default_rng(seed + 200_000)

```

```

adj = [set(a) for a in adj_init]
edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
n_tri = _count_triangles(adj, N)
S_curr = compute_S_canonical(adj, n_tri, N, mode, cfg)
n_steps = (bf + pf) * N
n_accept = 0
u_arr = rng.random(size=n_steps)
pool_arr = np.array(pool, dtype=np.int32) if pool else np.empty((0,2), dtype=
n_pool = len(pool_arr)
for step in range(n_steps):
    if not edge_list: break
    idx_out = int(rng.integers(len(edge_list)))
    u, v = edge_list[idx_out]
    e_in = None
    for _ in range(30):
        idx_in = int(rng.integers(n_pool))
        a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
        if b not in adj[a]:
            e_in = (a, b); break
    if e_in is None: continue
    a, b = e_in
    if len(adj[u]) <= 2 or len(adj[v]) <= 2:
        if not _fast_still_connected(adj, u, v, N):
            continue
    tri_lost = len(adj[u] & adj[v])
    adj[u].discard(v); adj[v].discard(u)
    adj[a].add(b); adj[b].add(a)
    tri_gained = len(adj[a] & adj[b])
    n_tri_new = n_tri - tri_lost + tri_gained
    S_new = compute_S_canonical(adj, n_tri_new, N, mode, cfg)
    dS = S_new - S_curr
    if dS >= 0 or u_arr[step] < math.exp(dS / tau):
        n_tri = n_tri_new
        S_curr = S_new
        n_accept += 1
        edge_list[idx_out] = (a, b)
    else:
        adj[u].add(v); adj[v].add(u)
        adj[a].discard(b); adj[b].discard(a)
accept_rate = n_accept / n_steps
n_tri_final = _count_triangles(adj, N)
return adj, S_curr, accept_rate, n_tri_final

def compute_dH(adj, N, cfg=CFG):
    """Méthodologie O2v4 exacte : greedy farthest-point, inner_fraction=0.70."""
    lcc, idx, _ = _extract_lcc(adj, N)
    M = len(lcc)

```

```

result = dict(d_H=None, d_H_std=None, d_H_flag=None, n_valid_src=0)
if M < 15:
    result["d_H_flag"] = "lcc_too_small"; return result
def bfs(src):
    d = {v: math.inf for v in lcc}; d[src] = 0; q = deque([src])
    while q:
        v = q.popleft()
        for w in adj[v]:
            if w in d and d[w] == math.inf:
                d[w] = d[v] + 1; q.append(w)
    return d
rng_s = np.random.default_rng(7)
sources = [lcc[rng_s.integers(M)]]
dts = bfs(sources[0])
while len(sources) < cfg["dH_n_sources"]:
    c = max((v for v in lcc if v not in sources),
            key=lambda v: dts.get(v, 0), default=None)
    if c is None: break
    sources.append(c); nd = bfs(c)
    for v in lcc:
        dts[v] = min(dts.get(v, math.inf), nd.get(v, math.inf))
slopes = []
for src in sources:
    d_map = bfs(src)
    fin = [d_map[v] for v in lcc if d_map[v] < math.inf]
    if not fin: continue
    rm = int(max(fin) * cfg["dH_inner_fraction"])
    if rm < cfg["dH_r_min_fit"] + 1: continue
    r_v = np.arange(cfg["dH_r_min_fit"], rm + 1)
    cnt = np.array([sum(1 for v in lcc if d_map[v] <= r) for r in r_v], float)
    vld = cnt >= cfg["dH_min_ball_count"]
    if vld.sum() < 3: continue
    try:
        s, _ = np.polyfit(np.log(r_v[vld]), np.log(cnt[vld]), 1)
        slopes.append(s)
    except: pass
result["n_valid_src"] = len(slopes)
if len(slopes) < cfg["dH_min_sources"]:
    result["d_H_flag"] = "insufficient_sources"; return result
result["d_H"] = float(np.mean(slopes))
result["d_H_std"] = float(np.std(slopes))
return result

def compute_ds_eigvals(adj, N):
    """Méthode alternative (claude) :  $N(\lambda) \propto \lambda^{(d_s/2)}$ ."""
    lcc, idx, _ = _extract_lcc(adj, N)
    M = len(lcc)

```

```

if M < 20: return None, None
L = np.zeros((M, M))
for v in lcc:
    iv = idx[v]
    nb = [w for w in adj[v] if w in idx]
    L[iv, iv] = len(nb)
    for w in nb:
        L[iv, idx[w]] = -1.0
evals = eigvalsh(L)
evals = np.sort(evals)
evals = evals[evals > 1e-9]
if len(evals) < 20: return None, None
lam_max = evals.max()
threshold = 0.15 * lam_max
mask = evals <= threshold
n_low = int(mask.sum())
if n_low < 10: n_low = max(20, len(evals) // 4)
lam_arr = evals[:n_low]
N_arr = np.arange(1, n_low + 1)
res = linregress(np.log(lam_arr), np.log(N_arr))
return float(2 * res.slope), float(res.rvalue ** 2)

if __name__ == "__main__":
    N = 500
    seeds = [0, 1, 2, 3, 4]
    print(f"=== Réplication O2v4 Phase A - N={N}, {len(seeds)} seeds ===")
    print(f"Paramètres : burn_in=100N, prod=200N, tau_eff=tau/N, beta={CFG['beta']}")
    print(f"Référence JSON Rudolphe (10 seeds) : d_H = 3.110 ± 0.156, n_tri ≈ 283")
    print()
    results = []
    t_total = time.time()
    for seed in seeds:
        t0 = time.time()
        pts, adj_init, pool, E0 = _build_initial(N, seed, CFG)
        K_init = sum(len(adj_init[i]) for i in range(N)) // 2
        lcc_init = _lcc_size(adj_init, N) / N
        if lcc_init < CFG["min_lcc_ratio"] or K_init != E0:
            print(f"  seed {seed}: rejected initial (lcc={lcc_init:.2f}, K={K_init:.2f})")
            continue
        adj, S, acc, n_tri = run_canonical_mcmc(pts, adj_init, pool, N, E0, "blob")
        t_mcmc = time.time() - t0
        t1 = time.time()
        dH_r = compute_dH(adj, N, CFG)
        t_dH = time.time() - t1
        t2 = time.time()
        d_s_eig, d_s_r2 = compute_ds_eigvals(adj, N)
        t_ds = time.time() - t2

```

```

results.append({
    'seed': seed, 'n_tri': n_tri, 'acc': acc,
    'd_H': dH_r['d_H'], 'd_H_std': dH_r['d_H_std'],
    'd_s_eig': d_s_eig, 'd_s_r2': d_s_r2,
})
dH_str = f"{dH_r['d_H']:.3f}±{dH_r['d_H_std']:.3f}" if dH_r['d_H'] is not
ds_str = f"{d_s_eig:.3f} (R²={d_s_r2:.3f})" if d_s_eig is not None else "
print(f"  seed {seed}: n_tri={n_tri:5d}  acc={acc:.3f}  "
      f"d_H={dH_str}  d_s_eig={ds_str}  "
      f"[mcmc {t_mcmc:.1f}s, dH {t_dH:.1f}s, ds {t_ds:.1f}s]")
if results:
    dH_vals = [r['d_H'] for r in results if r['d_H'] is not None]
    ds_vals = [r['d_s_eig'] for r in results if r['d_s_eig'] is not None]
    nt_vals = [r['n_tri'] for r in results]
    ac_vals = [r['acc'] for r in results]
    print()
    print("="*70)
    print(f"RÉSUMÉ N={N} (5 seeds, port fidèle O2v4)")
    print("="*70)
    print(f"  d_H mesuré          = {np.mean(dH_vals):.3f} ± {np.std(dH_vals):.3f}"
          f"  d_H référence JSON = 3.110 ± 0.156  ←  écart = {np.mean(dH_vals)-3.110:.3f}"
          f"  d_s eigvals       = {np.mean(ds_vals):.3f} ± {np.std(ds_vals):.3f}"
          f"  n_tri mesuré       = {np.mean(nt_vals):.0f} ± {np.std(nt_vals):.0f}"
          f"  n_tri référence    = 2839 ± 231      ←  écart = {np.mean(nt_vals)-2839:.0f}"
          f"  acceptance          = {np.mean(ac_vals):.3f}  (cible [0.20, 0.60])")
    print(f"\nTemps total : {time.time() - t_total:.1f}s")

```